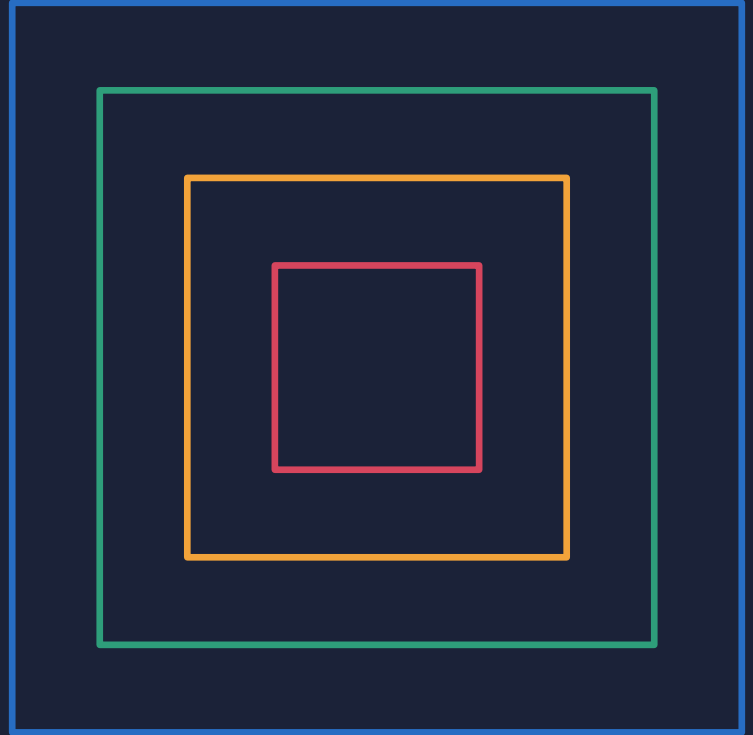


RECURSIVE DALAM R

Pertemuan 6

Algoritma dan Pemrograman

Oleh Weksi Budiaji & Ferdian Bangkit Wijaya

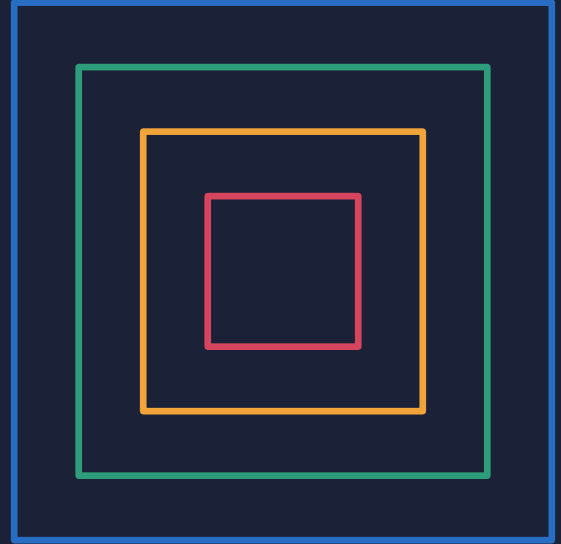


Daftar Materi

- 1 Definition
- 2 Factorial: Contoh Pertama Rekursi
- 3 Features
- 4 Types of Recursion
- 5 Example: Sum of Series
- 6 Rekursi Tanpa Base Case: Infinite Recursion
- 7 Ringkasan
- 8 Mini Kuis
- 9 Kunci Jawaban dan Pembahasan
- 10 Latihan Mandiri

1. Definition

Fungsi yang memanggil dirinya sendiri



Penjelasan: Apa itu Rekursi?

Rekursi adalah teknik pemrograman di mana sebuah fungsi **memanggil dirinya sendiri**, baik secara langsung maupun tidak langsung, untuk menyelesaikan suatu masalah. Rekursi merupakan alternatif dari loop (for, while, repeat) untuk melakukan pengulangan: daripada mengulang badan kode di dalam satu fungsi, rekursi "mengulang" dengan cara memanggil ulang fungsinya sendiri dengan input yang lebih sederhana / lebih kecil.

Ide dasarnya: sebuah masalah besar dipecah menjadi masalah yang **sama tetapi lebih kecil**, sampai akhirnya mencapai masalah paling sederhana yang bisa dijawab langsung tanpa pemanggilan lagi.

Dua Bagian Wajib Fungsi Rekursif

Bagian	Fungsi	Contoh (faktorial)
Base case (kasus dasar)	Kondisi paling sederhana yang jawabannya langsung diketahui, tanpa memanggil fungsi lagi. Inilah yang menghentikan rekursi.	<pre>if (x==0 x==1) return (1)</pre>
Recursive case (kasus rekursif)	Kondisi umum, di mana fungsi memanggil dirinya sendiri dengan input yang lebih dekat ke base case.	<pre>return (x * rec_fac(x-1))</pre>

PENTING

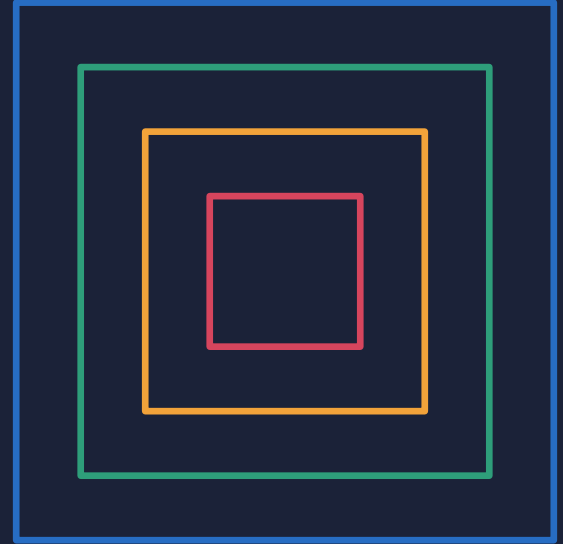
Setiap fungsi rekursif **wajib** memiliki base case yang pasti tercapai. Tanpa base case, atau jika input tidak pernah mendekati base case, fungsi akan memanggil dirinya sendiri **tanpa henti** hingga R kehabisan memori/stack dan menghentikan program dengan error (dibahas lebih lanjut di bagian 8).

Rekursi vs Loop

Aspek	Loop (for/while/repeat)	Rekursi
Cara mengulang	Blok kode diulang di dalam satu badan loop	Fungsi memanggil dirinya sendiri berulang kali
Penyimpanan status	Variabel biasa (mis. penghitung, akumulator)	Setiap pemanggilan menyimpan statusnya sendiri di call stack
Kapan cocok dipakai	Proses berulang yang sederhana, linear	Masalah yang secara alami terbagi menjadi sub-masalah serupa (mis. struktur pohon, data bersarang)
Risiko khas	Infinite loop bila lupa update	Infinite recursion / stack overflow bila base case tidak tercapai

2. Factorial: Contoh Pertama Rekursi

Contoh pertama rekursi di R



Definisi Faktorial

Faktorial dari bilangan bulat non-negatif x (ditulis $x!$) didefinisikan sebagai:

```
x! = x * (x-1) * (x-2) * ... * 1,   untuk x >= 1  
0! = 1   # didefinisikan begitu
```

Sudah rekursif secara matematis

Definisi ini sebenarnya sudah **rekursif** secara matematis, karena $x! = x * (x-1)!$. Inilah yang langsung diterjemahkan menjadi kode R.

Factorial

```
rec_fac <- function(x){  
  if(x==0 || x==1)  
  {  
    return(1)  
  }  
  else  
  {  
    return(x*rec_fac(x-1))  
  }  
}  
  
rec_fac(5)
```

Base case: if ($x==0$ || $x==1$) return(1). Baik 0! maupun 1! bernilai 1.

Recursive case: return($x*\text{rec_fac}(x-1)$). Untuk menghitung $x!$, fungsi memanggil dirinya sendiri untuk menghitung $(x-1)!$ lebih dulu, lalu mengalikannya dengan x .

Setiap pemanggilan $\text{rec_fac}(x-1)$ membuat input semakin kecil, sehingga cepat atau lambat akan mencapai base case $x==1$ (atau $x==0$).

Trace Pemanggilan `rec_fac(5)`

`rec_fac(5)`

`rec_fac(4)`

`rec_fac(3)`

`rec_fac(2)`

`rec_fac(1)`

Ada dua arah yang perlu ditelusuri: **arah memanggil** (turun, dari `rec_fac(5)` sampai base case) dan **arah mengembalikan nilai** (naik, dari base case kembali ke pemanggil awal). Setiap pemanggilan yang belum selesai **menunggu** hasil pemanggilan di bawahnya — inilah yang disebut **call stack**.

CARA MEMBAYANGKAN CALL STACK

Bayangkan lima kartu ditumpuk: kartu paling atas adalah `rec_fac(5)`, di bawahnya `rec_fac(4)`, dan seterusnya sampai `rec_fac(1)` paling bawah. R baru bisa **menghitung dan membuang** kartu paling bawah (base case) terlebih dahulu, lalu naik satu per satu ke atas sambil mengalikan, sampai kartu `rec_fac(5)` akhirnya bisa dihitung dan dibuang terakhir.

Arah Turun: Call Stack Terbentuk

Langkah	Pemanggilan	$x==0$ atau $x==1$?	Aksi
1	<code>rec_fac(5)</code>	FALSE	memanggil <code>rec_fac(4)</code> , menunggu hasilnya untuk dikalikan 5
2	<code>rec_fac(4)</code>	FALSE	memanggil <code>rec_fac(3)</code> , menunggu hasilnya untuk dikalikan 4
3	<code>rec_fac(3)</code>	FALSE	memanggil <code>rec_fac(2)</code> , menunggu hasilnya untuk dikalikan 3
4	<code>rec_fac(2)</code>	FALSE	memanggil <code>rec_fac(1)</code> , menunggu hasilnya untuk dikalikan 2
5	<code>rec_fac(1)</code>	TRUE	base case tercapai, langsung <code>return(1)</code> , tidak memanggil lagi

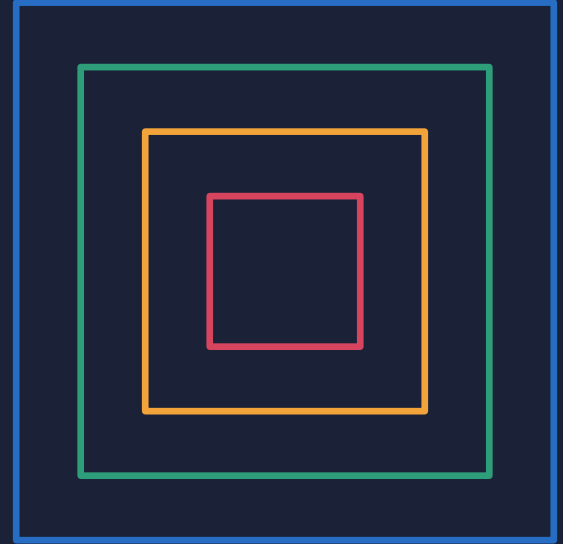
Arah Naik: Call Stack Dibongkar (Unwinding)

Langkah	Pemanggilan selesai	Nilai yang dikembalikan	Perhitungan
1	<code>rec_fac(1)</code>	1	base case
2	<code>rec_fac(2)</code>	2	$2 * \text{rec_fac}(1) = 2 * 1 = 2$
3	<code>rec_fac(3)</code>	6	$3 * \text{rec_fac}(2) = 3 * 2 = 6$
4	<code>rec_fac(4)</code>	24	$4 * \text{rec_fac}(3) = 4 * 6 = 24$
5	<code>rec_fac(5)</code>	120	$5 * \text{rec_fac}(4) = 5 * 24 = 120$

Output: [1] 120

3. Features

Mengapa dan kapan memakai rekursi



Features

1 Kode lebih ringkas dan jelas terbaca.

2 Solusi lebih sederhana

3 Mengekspresikan pemanggilan fungsi diri sendiri

Rekursi Bisa Membuat Kode Lebih Ringkas

Rekursi bisa membuat kode **lebih ringkas dan mudah dibaca** untuk masalah tertentu. Bandingkan versi rekursif faktorial dengan versi loop berikut, keduanya menghasilkan nilai yang sama:

```
# Versi loop
fac_loop <- function(x) {
  hasil <- 1
  for (i in 1:x) {
    hasil <- hasil * i
  }
  return(hasil)
}
```

```
# Versi rekursif
rec_fac <- function(x){
  if(x==0 || x==1) {return(1)}
  else {return(x*rec_fac(x-1))}
}
```

Faktorial vs Struktur Bersarang

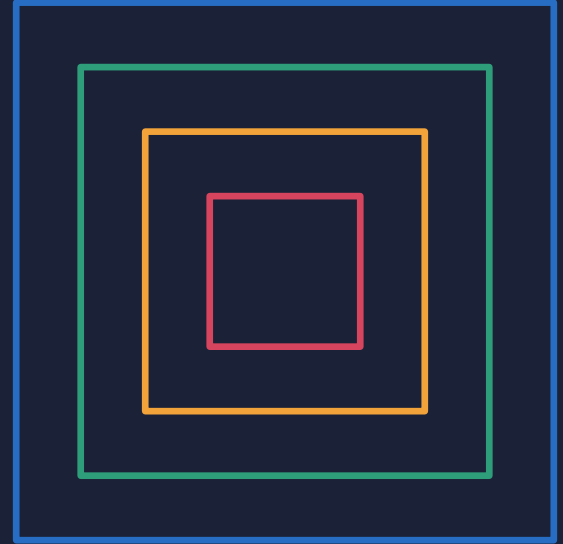
Untuk kasus faktorial, kedua versi sama-sama singkat. Namun untuk struktur data yang **bersarang atau bercabang** (pohon direktori, ekspresi matematika, jumlah elemen pada list di dalam list), versi rekursif biasanya jauh lebih ringkas daripada versi loop, karena loop harus mengurus tumpukan/antrian secara manual sedangkan rekursi mengandalkan call stack yang sudah disediakan R.

Kelebihan dan Kekurangan Rekursi

Kelebihan	Kekurangan
Kode sering lebih pendek dan mencerminkan definisi masalah secara langsung (mis. definisi faktorial matematis)	Setiap pemanggilan memakai memori (call stack), sehingga input besar dapat menyebabkan stack penuh
Cocok untuk struktur data rekursif seperti pohon, list bersarang, dan struktur folder	Bisa lebih lambat daripada loop karena overhead pemanggilan fungsi berulang kali
Alur pikir "pecah jadi sub-masalah lebih kecil" mudah dipahami untuk masalah tertentu	Jika base case salah/tidak tercapai, terjadi infinite recursion

4. Types of Recursion

Direct, indirect, mutual, nested, dan structural



Direct Recursion

Fungsi memanggil **dirinya sendiri secara langsung** di dalam badannya. Contoh `rec_fac()` pada bagian 2 adalah direct recursion: di dalam `rec_fac`, yang dipanggil adalah `rec_fac` itu sendiri.

```
rec_fac <- function(x){  
  if(x==0 || x==1) {return(1)}  
  else {return(x*rec_fac(x-1))} # <- memanggil dirinya sendiri  
}
```

Indirect Recursion

```
funcA <- function(n) {  
  if (n <= 0) return(0)  
  return(n + funcB(n-1)) # A memanggil B  
}  
funcB <- function(n) {  
  if (n <= 0) return(0)  
  return(n + funcA(n-1)) # B memanggil kembali A  
}  
  
funcA(4)
```

Fungsi A memanggil fungsi B, dan fungsi B (langsung atau melalui fungsi lain) memanggil kembali fungsi A. Rekursi tetap terjadi, tetapi **tidak langsung** — melalui perantara.

Trace: funcA(4) memanggil funcB(3), yang memanggil funcA(2), yang memanggil funcB(1), yang memanggil funcA(0) (base case, hasil 0). Nilai kemudian dijumlahkan sambil kembali naik.

Trace Indirect Recursion: funcA(4)

Panggilan	n	Base case?	Memanggil	Nilai kembali
funcA (4)	4	FALSE	funcB (3)	$4 + 6 = 10$
funcB (3)	3	FALSE	funcA (2)	$3 + 3 = 6$
funcA (2)	2	FALSE	funcB (1)	$2 + 1 = 3$
funcB (1)	1	FALSE	funcA (0)	$1 + 0 = 1$
funcA (0)	0	TRUE	–	0

Output: [1] 10

Mutual Recursion

```
is_even <- function(n) {  
  if (n == 0) return(TRUE)  
  else return(is_odd(n - 1))  
}  
is_odd <- function(n) {  
  if (n == 0) return(FALSE)  
  else return(is_even(n - 1))  
}  
  
is_even(4)
```

Kasus khusus indirect recursion: dua fungsi (atau lebih) **saling bergantung** dan saling memanggil untuk menyelesaikan satu tugas bersama. Contoh klasik: menentukan apakah suatu bilangan genap atau ganjil menggunakan dua fungsi yang saling memanggil.

Trace Mutual Recursion: `is_even(4)`

Panggilan	n	Base case?	Hasil
<code>is_even(4)</code>	4	FALSE	<code>is_odd(3)</code>
<code>is_odd(3)</code>	3	FALSE	<code>is_even(2)</code>
<code>is_even(2)</code>	2	FALSE	<code>is_odd(1)</code>
<code>is_odd(1)</code>	1	FALSE	<code>is_even(0)</code>
<code>is_even(0)</code>	0	TRUE	TRUE

Nilai TRUE dari `is_even(0)` diteruskan apa adanya ke setiap pemanggil di atasnya (tidak ada perhitungan tambahan seperti pada faktorial), sehingga `is_even(4)` akhirnya bernilai:

Output: [1] TRUE

Nested Recursion

Terjadi ketika **hasil dari satu pemanggilan rekursif dipakai sebagai argumen** untuk pemanggilan rekursif lain dari fungsi yang sama — argumennya "bersarang" (nested). Contoh sederhana berikut menunjukkan pola penulisannya:

```
nested_rec <- function(n) {  
  if (n <= 0) return(0)  
  return(nested_rec(nested_rec(n - 1))) # argumen berisi pemanggilan lagi  
}  
  
nested_rec(3)
```

Output: [1] 0

Ciri Khas Nested Recursion

1

Perhatikan pemanggilan `nested_rec(n - 1)` berada **di dalam** argumen pemanggilan `nested_rec(...)` yang lain — inilah ciri khas nested recursion.

2

Pada fungsi contoh ini, `nested_rec(k)` selalu berakhir bernilai 0 untuk $k \geq 0$ karena hasil dari `nested_rec(n-1)` akan terus mengecil hingga 0 lalu dibungkus lagi oleh `nested_rec(0) = 0`. Fungsi ini sengaja dibuat sederhana untuk memperlihatkan **pola/struktur** nested recursion, bukan untuk kegunaan praktis.

Trace Nested Recursion

Panggilan	Argumen dihitung dulu	Hasil argumen	Hasil akhir
<code>nested_rec(1)</code>	<code>nested_rec(0)</code>	0	<code>nested_rec(0) = 0</code>
<code>nested_rec(2)</code>	<code>nested_rec(1)</code>	0 (lihat baris di atas)	<code>nested_rec(0) = 0</code>
<code>nested_rec(3)</code>	<code>nested_rec(2)</code>	0 (lihat baris di atas)	<code>nested_rec(0) = 0</code>

CONTOH NESTED RECURSION YANG LEBIH TERKENAL

Fungsi McCarthy 91 adalah contoh nested recursion klasik yang lebih bermakna secara matematis, tetapi perilakunya lebih rumit untuk ditelusuri di tahap awal. Untuk kebutuhan kuliah ini, cukup pahami **pola penulisannya**: hasil satu pemanggilan rekursif menjadi argumen pemanggilan rekursif lain dari fungsi yang sama.

Structural Recursion

Rekursi yang alurnya mengikuti **struktur data**: data dipecah menjadi bagian pertama (head) dan sisanya (tail), lalu bagian pertama diproses dan sisanya diserahkan ke pemanggilan berikutnya. Cocok untuk vektor, list, atau struktur data lain yang bisa dipecah menjadi "bagian depan" dan "sisanya".

```
vec_sum <- function(v) {  
  if (length(v) == 0) return(0) # base case: vektor kosong  
  else return(v[1] + vec_sum(v[-1])) # elemen pertama +  
  sisa vektor  
}  
  
vec_sum(c(4, 7, 1, 9))
```

Base case: **struktur data habis** (length(v) == 0), bukan sebuah angka seperti pada faktorial.

v[1] : elemen pertama (bagian yang diproses sekarang).

v[-1] : seluruh elemen **kecuali** elemen pertama (sisa struktur yang diserahkan ke pemanggilan berikutnya, semakin lama semakin pendek).

Trace Structural Recursion: vec_sum

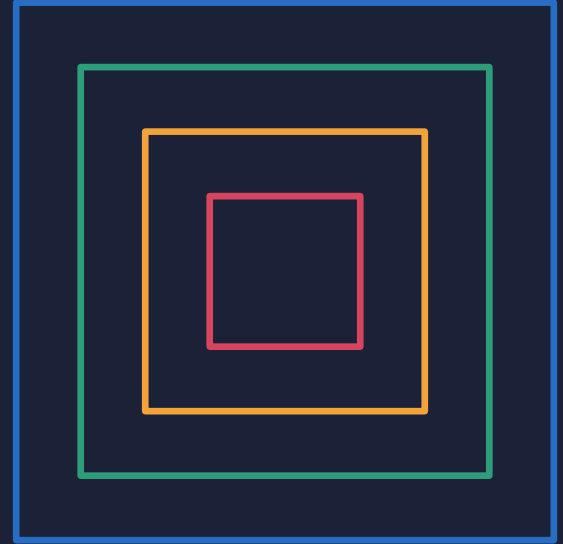
Panggilan	v	length(v)==0?	v[1]	v[-1]
1	c(4, 7, 1, 9)	FALSE	4	c(7, 1, 9)
2	c(7, 1, 9)	FALSE	7	c(1, 9)
3	c(1, 9)	FALSE	1	c(9)
4	c(9)	FALSE	9	c()
5	c()	TRUE	-	-

Arah naik: `vec_sum(c())=0`, lalu $9+0=9$, $1+9=10$, $7+10=17$, $4+17=21$.

Output: [1] 21

5. Example: Sum of Series Using Recursion

Jumlah kuadrat deret bilangan



Materi Slide: Example

Example: Sum of Series Using Recursion Recursion in R is most useful for finding the sum of self-repeating series. In this example, we will find the sum of squares of a given series of numbers.

$$\text{Sum} = 1^2 + 2^2 + \dots + N^2$$

Menyusun Fungsi Rekursif

Menulis fungsi rekursif: tentukan base case, lalu tentukan recursive case.

1

Tentukan base case. Deret berhenti ketika N mencapai 0 — jumlah dari deret kosong adalah 0.

2

Tentukan recursive case. Jumlah sampai N sama dengan N^2 ditambah jumlah sampai N-1. Ini persis definisi matematis $\text{Sum}(N) = N^2 + \text{Sum}(N-1)$.

```
sum_sq <- function(n) {  
  if (n <= 0) {  
    return(0)  
  } else {  
    return(n^2 + sum_sq(n-1))  
  }  
}  
  
sum_sq(4)
```

Trace sum_sq(4): Arah Turun

Langkah	Pemanggilan	$n \leq 0$?	Aksi
1	sum_sq (4)	FALSE	memanggil sum_sq(3), menunggu untuk ditambah $4^2=16$
2	sum_sq (3)	FALSE	memanggil sum_sq(2), menunggu untuk ditambah $3^2=9$
3	sum_sq (2)	FALSE	memanggil sum_sq(1), menunggu untuk ditambah $2^2=4$
4	sum_sq (1)	FALSE	memanggil sum_sq(0), menunggu untuk ditambah $1^2=1$
5	sum_sq (0)	TRUE	base case, langsung return(0)

Trace sum_sq(4): Arah Naik

Langkah	Pemanggilan selesai	Nilai dikembalikan	Perhitungan
6	sum_sq(0)	0	base case
7	sum_sq(1)	1	$1^2 + 0 = 1$
8	sum_sq(2)	5	$2^2 + 1 = 4 + 1 = 5$
9	sum_sq(3)	14	$3^2 + 5 = 9 + 5 = 14$
10	sum_sq(4)	30	$4^2 + 14 = 16 + 14 = 30$

Cocok dengan perhitungan langsung: $1^2 + 2^2 + 3^2 + 4^2 = 1 + 4 + 9 + 16 = 30$.

Output: [1] 30

Variasi: N Bisa Juga dari 1 Menaik

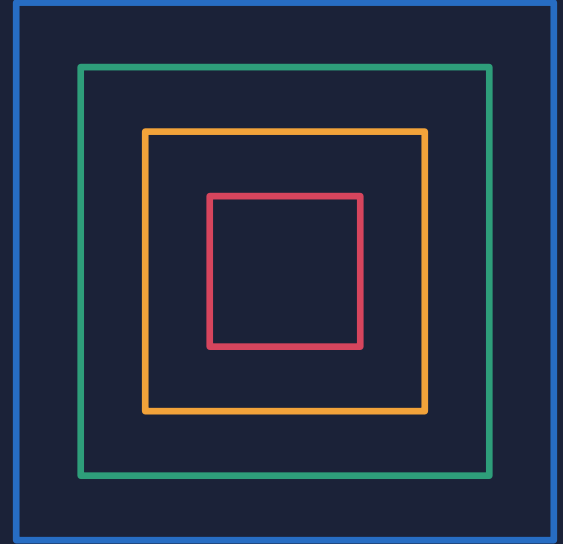
Versi di atas menghitung deret dari n **turun** ke 1 (top-down). Alternatifnya, tulis fungsi dengan dua parameter agar deret dihitung **naik** dari 1 ke n:

```
sum_sq2 <- function(i, n) {  
  if (i > n) return(0)  
  else return(i^2 + sum_sq2(i+1, n))  
}  
  
sum_sq2(1, 4) # mulai dari i=1 sampai n=4
```

Hasilnya sama, 30, hanya urutan penjumlahannya yang berbeda arah.

6. Rekursi Tanpa Base Case: Infinite Recursion

Ketika rekursi tidak pernah berhenti



Infinite Recursion

Sama seperti loop yang bisa menjadi infinite loop, fungsi rekursif yang **tidak memiliki base case**, atau base case-nya **tidak pernah tercapai**, akan memanggil dirinya sendiri tanpa henti.

```
# JANGAN DIJALANKAN TANPA BATASAN — hanya untuk
ilustrasi
bahaya <- function(x) {
  return(x * bahaya(x - 1)) # tidak ada base
case!
}
```

Output: Error: C stack usage too close to the limit

Karena tidak ada if yang menghentikan pemanggilan, bahaya(5) akan terus memanggil bahaya(4), bahaya(3), bahaya(2), bahaya(1), bahaya(0), bahaya(-1), dan seterusnya tanpa batas. Setiap pemanggilan menambah satu "kartu" ke call stack, sampai akhirnya R kehabisan ruang dan menghentikan proses dengan pesan galat, misalnya:

Cara Menghindarinya

1

Pastikan **selalu ada** kondisi base case yang bisa dijangkau.

2

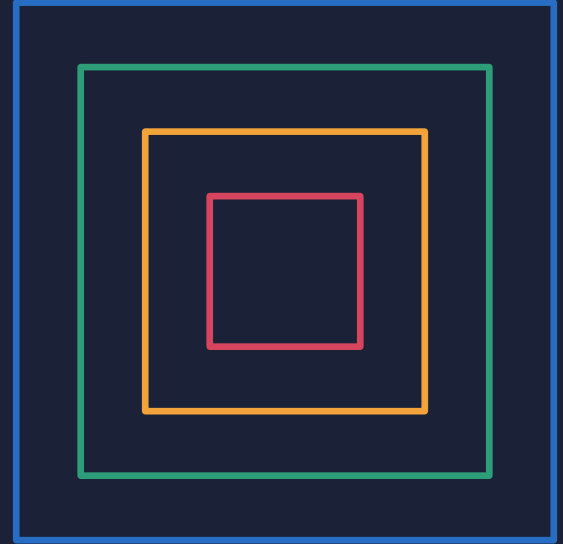
Pastikan setiap pemanggilan rekursif membawa input yang **bergerak mendekati** base case (mis. $x-1$ mendekati 0, $v[-1]$ membuat vektor makin pendek).

3

Untuk data yang sangat besar/dalam, pertimbangkan versi loop, atau naikkan batas kedalaman rekursi dengan `options(expressions = 5000)` (nilai bawaan R adalah 5000; batas ini tetap ada meski dinaikkan).

7. Ringkasan

Poin-poin utama materi



Ringkasan (1/2)

1

Rekursi adalah fungsi yang memanggil dirinya sendiri untuk menyelesaikan masalah dengan memecahnya menjadi sub-masalah yang lebih kecil.

2

Setiap fungsi rekursif membutuhkan **base case** (berhenti) dan **recursive case** (memanggil lagi dengan input lebih kecil).

3

Lima jenis rekursi: **direct** (memanggil diri sendiri langsung), **indirect** (melalui fungsi lain), **mutual** (dua fungsi saling bergantung), **nested** (hasil satu pemanggilan menjadi argumen pemanggilan lain), dan **structural** (mengikuti struktur data, mis. head + tail).

Ringkasan (2/2)

4

Menelusuri rekursi dibantu dengan membayangkan **call stack**: arah turun membentuk tumpukan pemanggilan yang menunggu, arah naik membongkar tumpukan sambil menghitung hasil.

5

Tanpa base case yang tercapai, rekursi menjadi infinite recursion dan berhenti karena stack penuh.

6

Faktorial (`rec_fac`) dan jumlah kuadrat (`sum_sq`) adalah dua contoh khas rekursi numerik yang mengikuti definisi matematis langsung.